HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN I

ViMind - Hệ thống Multi-Agent RAG hỗ trợ tâm lý người Việt

Giảng viên: Kim Ngọc Bách
Họ tên sinh viên: Hoàng Chí Hiển
Mã sinh viên: B23DCCE028
Lớp: D23CQCE04-B

Hà Nội – 2026

1. Tổng quan dự án

1.1. Bối cảnh và lý do chọn đề tài

1.1.1. Bối cảnh xã hội: Nhu cầu cấp thiết về chăm sóc sức khỏe tinh thần tại Việt Nam

Trong bối cảnh đô thị hóa nhanh chóng và áp lực từ môi trường học tập, công việc hiện đại, tỷ lệ người gặp các vấn đề về sức khỏe tinh thần (như trầm cảm, rối loạn âu lo, stress mãn tính) tại Việt Nam đang gia tăng mạnh mẽ. Tuy nhiên, việc tiếp cận các dịch vụ chăm sóc tâm lý chuyên nghiệp đang vấp phải ba rào cản lớn:

- **Sự thiếu hụt nguồn lực chuyên môn:** Số lượng bác sĩ tâm thần và chuyên gia tâm lý lâm sàng tại Việt Nam còn mỏng, dẫn đến tình trạng quá tải tại các cơ sở y tế chuyên khoa.
- **Rào cản kinh tế và thời gian:** Chi phí cho các buổi tham vấn tâm lý 1:1 thường cao so với thu nhập trung bình, đồng thời lịch trình không linh hoạt khiến nhiều người khó duy trì trị liệu dài hạn.
- **Rào cản tâm lý và định kiến xã hội:** Tâm lý e ngại bị đánh giá khiến nhiều người bệnh chọn cách che giấu, tự chịu đựng thay vì tìm kiếm sự trợ giúp chuyên môn trong giai đoạn khởi phát.
  Do đó, việc phát triển một hệ thống hỗ trợ tâm lý có khả năng ẩn danh, hoạt động 24/7 và dễ dàng tiếp cận là một nhu cầu xã hội cấp thiết.

1.1.2. Bối cảnh công nghệ: Giới hạn của các hệ thống AI và RAG truyền thống trong miền dữ liệu tâm lý

Sự bùng nổ của các mô hình ngôn ngữ lớn (LLMs) đã thúc đẩy các ứng dụng AI đàm thoại. Để giải quyết bài toán "ảo giác thông tin" (hallucination) của LLMs, kỹ thuật Retrieval-Augmented Generation (RAG) được ứng dụng rộng rãi nhằm truy xuất tri thức từ cơ sở dữ liệu nội bộ. Tuy nhiên, khi áp dụng vào miền dữ liệu tâm lý học bằng Tiếng Việt, hệ thống LLM cơ bản và RAG Single-Agent bộc lộ các yếu điểm cốt lõi:

- **Thiếu khả năng phân tách luồng nhận thức:** Tư vấn tâm lý không đơn thuần là hỏi - đáp kiến thức. Nó đòi hỏi quy trình phức tạp: lắng nghe, phân tích cảm xúc, xác định vấn đề cốt lõi, truy xuất phương pháp can thiệp phù hợp, và phản hồi với giọng văn thấu cảm. Một RAG pipeline tuyến tính truyền thống không thể xử lý đồng thời và hiệu quả khối lượng công việc đa nhiệm này.

- **Hạn chế trong suy luận ngữ cảnh sâu và đa chiều:** Các câu hỏi của người dùng gặp vấn đề tâm lý thường mơ hồ, chứa đựng cảm xúc tiêu cực ẩn lấp và bối cảnh phân mảnh. Single-Agent RAG thường chỉ trích xuất bề mặt từ khóa (keyword-based hoặc semantic vector matching cơ bản), dẫn đến các câu trả lời mang tính rập khuôn, lý thuyết, thiếu sự cá nhân hóa và thấu cảm cần thiết.
- **Rủi ro về an toàn:** Việc không có cơ chế độc lập để kiểm duyệt và đánh giá mức độ nghiêm trọng của người dùng (ví dụ: ý định tự hại, khủng hoảng cấp tính) khiến các hệ thống AI đơn luồng trở nên nguy hiểm nếu đưa ra lời khuyên sai lệch.
- **Khoảng trống trong xử lý ngôn ngữ tự nhiên Tiếng Việt:** Ngữ cảnh tâm lý học bằng Tiếng Việt có độ phức tạp cao do đặc thù văn hóa, cách diễn đạt ẩn ý và từ lóng. Các mô hình mã nguồn mở hiện nay thiếu sự tinh chỉnh sâu trên tập dữ liệu đặc thù này.
- **Sự phụ thuộc vào LLMs thông qua API bên thứ ba:** Phần lớn các hệ thống AI hiện nay vận hành bằng cách gọi lõi LLMs thông qua API từ các nhà cung cấp bên thứ ba. Cách tiếp cận này giúp triển khai nhanh nhưng đồng thời đặt ra nhiều hạn chế: khó kiểm soát hoàn toàn quá trình suy luận nội tại của mô hình, phụ thuộc vào chính sách và giới hạn tài nguyên của nhà cung cấp, chi phí vận hành cao khi mở rộng quy mô, và hạn chế khả năng tùy biến sâu theo miền dữ liệu chuyên biệt. Trong bối cảnh xử lý dữ liệu tâm lý nhạy cảm, sự phụ thuộc này cũng đặt ra thách thức liên quan đến bảo mật và quyền riêng tư dữ liệu người dùng.

Từ những hạn chế trên, có thể thấy rằng việc phát triển một kiến trúc nâng cao, có khả năng phân vai, kiểm soát và tối ưu hóa theo miền dữ liệu tiếng Việt là một yêu cầu cần thiết thay vì chỉ dựa vào các hệ thống RAG truyền thống hoặc LLM được cung cấp sẵn qua API.

1.1.3. Lý do đề xuất hệ thống ViMind

Trước những hạn chế trên, đề tài "**ViMind – Hệ thống Multi-Agent RAG hỗ trợ tâm lý người Việt**" được đề xuất như một hướng tiếp cận mở rộng và toàn diện hơn. Việc chuyển dịch từ mô hình Single-Agent sang kiến trúc Multi-Agent RAG không chỉ là thay đổi về cấu trúc kỹ thuật, mà còn là sự điều chỉnh về tư duy thiết kế hệ thống theo hướng chuyên môn hóa và cộng tác hóa.

Trước hết, hệ thống được xây dựng dựa trên nguyên tắc chuyên môn hóa tác vụ. Thay vì để một mô hình duy nhất đảm nhiệm toàn bộ quy trình xử lý,

ViMind phân tách thành nhiều tác tử với vai trò riêng biệt. Sự phân vai này cho phép từng thành phần được tối ưu hóa theo nhiệm vụ cụ thể, từ đó nâng cao độ chính xác và tính ổn định của toàn hệ thống.

Bên cạnh đó, kiến trúc Multi-Agent tạo điều kiện để triển khai các cơ chế như self-correction và suy luận đa bước. Các tác tử không hoạt động tách rời mà có thể trao đổi, đánh giá chéo và điều chỉnh kết quả của nhau trước khi tổng hợp phản hồi cuối cùng. Nhờ vậy, thông tin cung cấp cho người dùng có cơ sở vững chắc hơn, tiệm cận với các nguyên tắc thực hành tâm lý dựa trên các tài liệu tin cậy, đồng thời giảm thiểu nguy cơ đưa ra phản hồi thiếu chính xác hoặc thiếu cân nhắc.

Ngoài yếu tố kiến trúc, đề tài còn chú trọng đến việc xây dựng và chuẩn hóa bộ dữ liệu tâm lý tiếng Việt. Trong bối cảnh phần lớn tài nguyên AI tập trung vào tiếng Anh, việc phát triển một nền tảng dữ liệu đặc thù cho tiếng Việt không chỉ giúp cải thiện chất lượng truy xuất tri thức mà còn góp phần đảm bảo tính phù hợp về ngôn ngữ và văn hóa. Hệ thống sử dụng một mô hình LLM được fine-tune trên chính bộ dữ liệu tâm lý tự xây dựng này, nhằm tăng cường khả năng hiểu ngữ cảnh, sắc thái cảm xúc và cách diễn đạt đặc trưng của người Việt. Cách tiếp cận này giúp nâng cao mức độ cá nhân hóa phản hồi và giảm sự phụ thuộc hoàn toàn vào các mô hình tổng quát bên ngoài. Điều này cho phép hệ thống không dừng lại ở việc cung cấp thông tin, mà còn tương tác theo cách gần gũi, tự nhiên và phù hợp với bối cảnh tâm lý của người Việt.

Tóm lại, ViMind không chỉ là một giải pháp công nghệ nhằm tối ưu hóa kiến trúc RAG, mà còn là một bước tiến quan trọng trong việc ứng dụng AI để tạo ra một công cụ hỗ trợ sơ cứu tâm lý an toàn, chính xác và thấu cảm cho cộng đồng.

1.2. Tính ứng dụng của đề tài

Hệ thống ViMind được thiết kế với định hướng ứng dụng thực tiễn cao, tập trung vào các môi trường triển khai sau:

- **Hỗ trợ tư vấn sơ cấp đa nền tảng:** Triển khai linh hoạt dưới dạng ứng dụng web và chatbot, cung cấp dịch vụ hỗ trợ tâm lý sơ cấp 24/7, giúp người dùng dễ dàng tiếp cận một cách ẩn danh.
- **Cung cấp tri thức chuẩn xác:** Trích xuất và phản hồi thông tin tâm lý học hoàn toàn dựa trên cơ sở dữ liệu và cẩm nang chuyên môn đã qua kiểm duyệt, loại bỏ rủi ro sai lệch thông tin thường gặp ở các mô hình AI thông thường.

- **Sàng lọc và định hướng chuyên sâu:** Tích hợp cơ chế tự động phân tích rủi ro. Khi phát hiện các dấu hiệu nguy hiểm hoặc khủng hoảng tâm lý nghiêm trọng, hệ thống sẽ lập tức cảnh báo và cung cấp thông tin liên hệ, định hướng người dùng đến các chuyên gia tâm lý hoặc cơ sở y tế kịp thời.
- **Triển khai trong môi trường giáo dục:** Ứng dụng trực tiếp vào môi trường đại học (ví dụ: triển khai mạng lưới hỗ trợ nội bộ cho sinh viên tại Học viện Công nghệ Bưu chính Viễn thông) để tạo ra một kênh tham vấn thường trực, giúp sinh viên giải tỏa áp lực học tập và các vấn đề định hướng cá nhân.
- **Tích hợp vào quản trị nhân sự doanh nghiệp:** Có khả năng mở rộng để tích hợp như một module hỗ trợ trong các hệ thống quản trị nhân sự, giúp doanh nghiệp theo dõi, chăm sóc sức khỏe tinh thần và duy trì hiệu suất làm việc cho người lao động.
- **Định vị sản phẩm rõ ràng:** Hệ thống được xác định rõ không nhằm mục đích thay thế quy trình chẩn đoán y khoa hay vai trò của chuyên gia tâm lý thực thụ. ViMind đóng vai trò là công cụ hỗ trợ tiếp cận ban đầu, nâng cao nhận thức cá nhân và chuẩn bị tâm lý trước khi người dùng gặp chuyên gia.

1.3. Giá trị học thuật

Đề tài đóng góp trực tiếp vào lĩnh vực nghiên cứu Trí tuệ nhân tạo và Xử lý ngôn ngữ tự nhiên, với các điểm nhấn khoa học sau:

- **Đề xuất kiến trúc Multi-Agent RAG chuyên biệt:** Thiết kế và tối ưu hóa hệ thống RAG đa tác tử dành riêng cho miền dữ liệu tâm lý Tiếng Việt. Kiến trúc này hỗ trợ khả năng tích hợp linh hoạt với các mô hình ngôn ngữ nội bộ được tinh chỉnh và có thể triển khai thực tế trên các nền tảng máy chủ hiệu năng cao như vLLM.
- **Cải tiến chiến lược truy xuất ngữ nghĩa tiếng Việt:** Đưa ra các giải pháp tinh chỉnh chiến lược embedding và truy xuất tri thức (retrieval) để xử lý triệt để các đặc thù về ngữ pháp, từ vựng biểu đạt cảm xúc và ngữ cảnh ẩn ý trong tiếng Việt.
- **Phát triển mô hình LLM fine-tune chuyên biệt cho lĩnh vực tâm lý:** Xây dựng và tinh chỉnh một mô hình ngôn ngữ dựa trên bộ dữ liệu tâm lý tiếng Việt được thu thập và chuẩn hóa có hệ thống. Quá trình fine-tuning tập trung vào việc cải thiện khả năng nhận diện sắc thái cảm xúc, hiểu ngữ cảnh ẩn ý, và tạo phản hồi thấu cảm theo chuẩn mực tham vấn tâm lý. Mô hình chuyên biệt này giúp giảm thiểu tính rập

khuôn của LLM tổng quát, đồng thời tăng độ phù hợp khi xử lý các tình huống nhạy cảm liên quan đến sức khỏe tinh thần.
- **Xây dựng pipeline đánh giá cho RAG agents:** Đề xuất bộ tiêu chí đánh giá kết hợp giữa các độ đo kỹ thuật hệ thống RAG.
- **Mở rộng hướng nghiên cứu AI trong y tế:** Tạo tiền đề khoa học và cung cấp tài liệu tham khảo thực nghiệm giá trị cho cộng đồng nghiên cứu AI tại Việt Nam trong việc ứng dụng công nghệ xử lý ngôn ngữ tự nhiên vào lĩnh vực sức khỏe tinh thần.

2. Nền tảng kiến thức

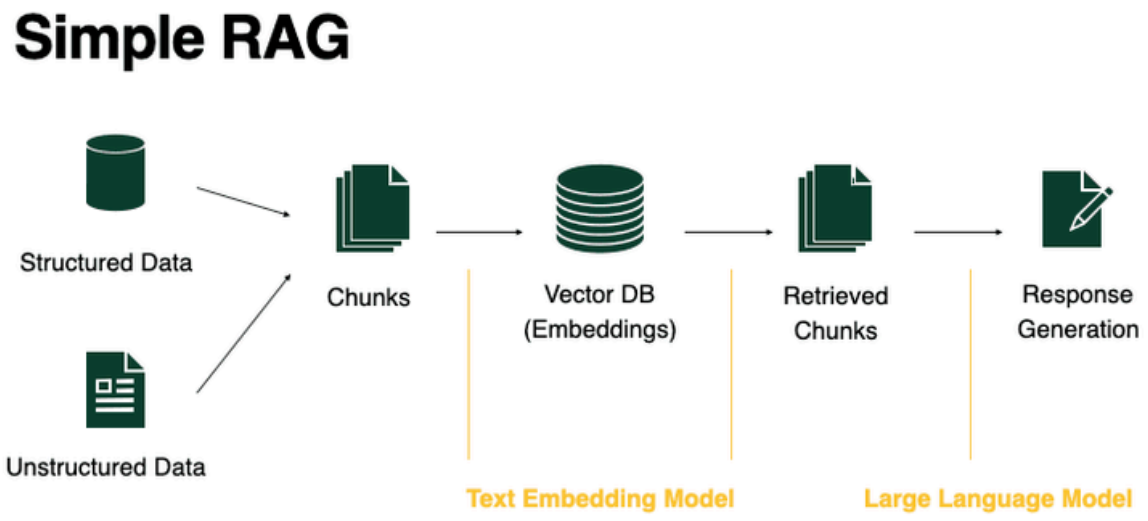
2.1. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) là một phương pháp kết hợp năng lực tạo sinh của các mô hình ngôn ngữ lớn (LLMs) với hệ thống truy xuất thông tin truyền thống. Kiến trúc này được đề xuất lần đầu bởi Lewis và các cộng sự (2020) [1] nhằm giải quyết các bài toán xử lý ngôn ngữ tự nhiên đòi hỏi lượng tri thức chuyên sâu.

Mục tiêu cốt lõi của RAG là khắc phục hiện tượng "ảo giác thông tin" (hallucination) của LLMs. Thay vì phụ thuộc hoàn toàn vào tham số mô hình đã được huấn luyện tĩnh, RAG cấp cho mô hình quyền truy cập vào một cơ sở dữ liệu tri thức bên ngoài theo thời gian thực.

Một quy trình RAG tiêu chuẩn bao gồm hai thành phần hoạt động tuần tự:

- **Mô hình truy xuất (Retriever):** Có nhiệm vụ tìm kiếm các tài liệu liên quan nhất từ cơ sở dữ liệu ngoại vi. Cụ thể, các đoạn văn bản (chunks) và câu truy vấn của người dùng đều được chuyển đổi thành các biểu diễn không gian vector thông qua một mô hình embedding. Mức độ liên quan giữa truy vấn và tài liệu được tính toán dựa trên các phép đo khoảng cách, phổ biến nhất là cosine similarity. Retriever sẽ trả về top-k tài liệu có điểm số tương đồng cao nhất.
- **Mô hình tạo sinh (Generator):** Thường là một mô hình Sequence-to-Sequence hoặc một LLM (như GPT, LLaMA). Generator sẽ nhận đầu vào là một câu lệnh đã được bổ sung đoạn văn bản truy xuất từ retriever. Dựa trên dữ kiện thực tế được cung cấp trong prompt, mô hình tiến hành suy luận và sinh ra câu trả lời cuối cùng.



Hình 1. Kiến trúc RAG cơ bản

RAG cho phép hệ thống ViMind cung cấp thông tin dựa trên các cẩm nang trị liệu tâm lý Tiếng Việt chuẩn xác mà không cần tiêu tốn chi phí để huấn luyện lại toàn bộ tham số của LLM.

Tuy nhiên, các hệ thống RAG truyền thống thường vận hành theo một quy trình khá đơn giản, trong đó quá trình truy xuất và sinh phản hồi diễn ra tuần tự và tương đối khép kín. Cách tiếp cận này có thể đáp ứng tốt các truy vấn mang tính thông tin đơn giản, nhưng dần bộc lộ giới hạn khi phải xử lý những vấn đề tâm lý phức tạp và giàu chiều sâu ngữ cảnh.

Hỗ trợ tâm lý không chỉ là việc trích xuất một tài liệu phù hợp rồi chuyển hóa thành câu trả lời. Quá trình này đòi hỏi khả năng “định tuyến nhận thức”, bao gồm phân tích sắc thái cảm xúc, nhận diện dấu hiệu rủi ro như xu hướng tự hại, đồng thời suy luận nhiều bước để tìm ra nguyên nhân cốt lõi của vấn đề. Trong bối cảnh đó, một pipeline tuyến tính khó có thể đảm nhiệm đồng thời các tầng xử lý khác nhau với mức độ chính xác và độ tin cậy cao.

Chính sự thiếu vắng cơ chế tư duy song song và đối soát chéo giữa các bước xử lý đã đặt ra yêu cầu cần một kiến trúc linh hoạt và cộng tác hơn. Đây cũng là cơ sở để đề tài định hướng nâng cấp từ RAG truyền thống sang kiến trúc Multi-Agent RAG, nhằm tạo ra một hệ thống có khả năng phân vai, kiểm chứng lẫn nhau và suy luận sâu hơn trong miền dữ liệu tâm lý.

2.2. LLM-based Multi-Agent Systems

2.2.1. Autonomous Agent

Trong ngữ cảnh của trí tuệ nhân tạo hiện đại, một tác tử (agent) không chỉ là một mô hình học máy phản hồi thụ động mà là một thực thể có khả năng tự trị ở mức độ nhất định. Theo định nghĩa của Wang và cộng sự (2023) [2], một tác tử tự trị dựa trên LLM được cấu trúc dựa trên ba thành phần cốt lõi:

- **Bộ não (Brain):** Sử dụng LLM làm lõi điều khiển trung tâm để phân tích ngôn ngữ tự nhiên, lập luận và lên kế hoạch (planning). Bộ não này có khả năng phân rã một nhiệm vụ phức tạp thành các bước nhỏ hơn.
- **Bộ nhớ (Memory):** Bao gồm bộ nhớ ngắn hạn và bộ nhớ dài hạn, cho phép tác tử lưu trữ và truy xuất thông tin từ các tương tác trong quá khứ.
- **Hành động và Công cụ:** Tác tử có khả năng thực thi các hành động cụ thể thông qua việc gọi các API hoặc các công cụ ngoại vi (như tìm kiếm web, truy vấn cơ sở dữ liệu) để hoàn thành các bước trong kế hoạch.

2.2.2. Hệ thống Đa tác tử (Multi-Agent System - MAS)

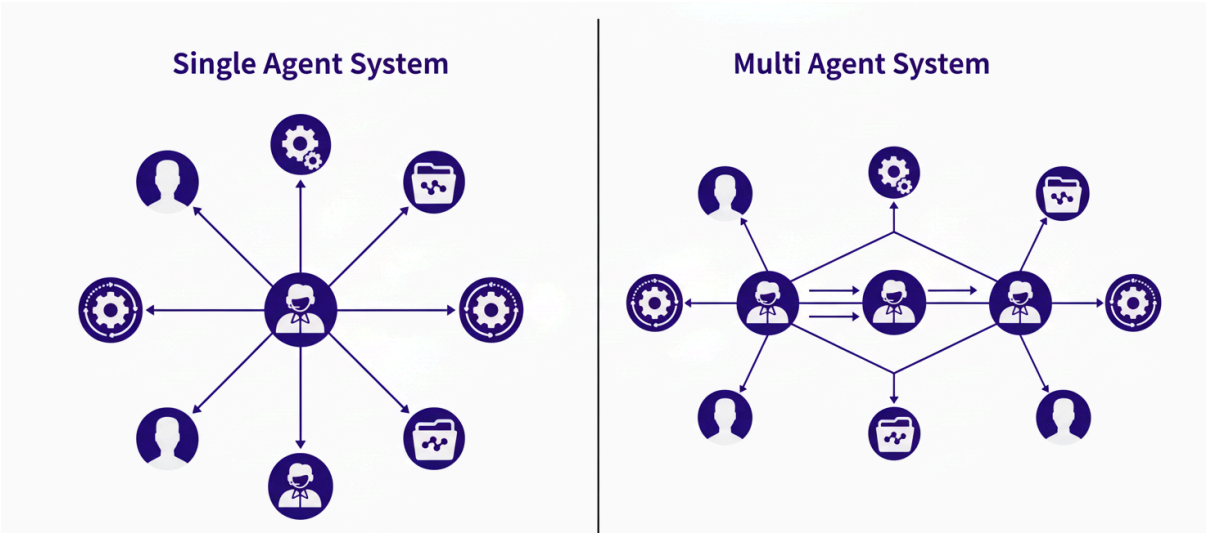
Hệ thống đa tác tử là một môi trường tính toán phân tán nơi nhiều tác tử độc lập tương tác và cộng tác với nhau để giải quyết các bài toán phức tạp vượt quá khả năng xử lý của một tác tử đơn lẻ. Việc kết hợp nhiều LLM agents trong một hệ thống đã được Wu và cộng sự (2023) [3] mô hình hóa thông qua các nền tảng như AutoGen, cho phép thiết lập các luồng hội thoại đa dạng giữa các tác tử.

Kiến trúc phân rã chức năng là cơ chế hoạt động chủ đạo trong MAS. Thay vì sử dụng một LLM duy nhất để xử lý toàn bộ khối lượng công việc từ đầu đến cuối, MAS phân chia hệ thống thành các tác tử chuyên biệt, mỗi tác tử được gán một vai trò và bộ công cụ riêng biệt. Các mô hình tương tác phổ biến bao gồm:

- **Hợp tác theo chuỗi:** Tác tử A xử lý đầu vào và truyền kết quả đầu ra làm đầu vào cho tác tử B.
- **Kiểm tra chéo và tranh luận:** Các tác tử sinh ra nhiều luồng kết quả khác nhau, sau đó một tác tử giám sát sẽ đánh giá và chọn lọc hoặc yêu cầu các tác tử trước đó điều chỉnh lại trước khi đưa ra kết quả cuối cùng.

Tổng kết lại, Kiến trúc Multi-Agent cho phép hệ thống ViMind phân chia các tác vụ phức tạp trong tư vấn tâm lý cho nhiều tác tử chuyên biệt xử lý đồng

thời. Việc ứng dụng hệ thống đa tác tử giúp tự động hóa quá trình kiểm tra chéo thông tin, tối ưu hóa khả năng suy luận đa chiều và đảm bảo mức độ an toàn, chuẩn xác cao nhất cho câu trả lời trước khi phản hồi người dùng, khắc phục hoàn toàn sự quá tải hoặc giới hạn ngữ cảnh của một mô hình ngôn ngữ đơn lẻ.



Hình 2. Hệ thống đa tác tử (MAS)

2.3. LLM finetuning

2.3.1. Tổng quan về LLM Finetuning trong miền dữ liệu đặc thù

Các mô hình ngôn ngữ lớn ở dạng cơ sở (Base LLMs) thường được huấn luyện trên khối lượng dữ liệu khổng lồ, đa miền và đa ngôn ngữ, với mục tiêu tối ưu hóa khả năng dự đoán token tiếp theo ở mức tổng quát. Tuy nhiên, chính đặc tính tổng quát này khiến chúng chưa thực sự phù hợp khi triển khai trong các miền chuyên biệt như tâm lý học, đặc biệt là trong bối cảnh tiếng Việt.

Thứ nhất, Base LLMs thường thiếu khả năng tuân thủ chặt chẽ các instruction format hoặc quy tắc phản hồi nghiêm ngặt nếu không được căn chỉnh bổ sung. Trong miền hỗ trợ tâm lý, phản hồi không chỉ cần đúng về mặt nội dung mà còn phải tuân theo cấu trúc nhất định: giọng văn thấu cảm, tránh phán xét, không đưa ra chẩn đoán y khoa vượt thẩm quyền và luôn kèm theo khuyến nghị tìm đến chuyên gia khi cần thiết. Nếu không được tinh chỉnh, mô hình cơ sở có thể tạo ra phản hồi mang tính suy đoán quá mức hoặc thiếu kiểm soát về ngữ điệu.

Thứ hai, các mô hình nền tảng phần lớn được huấn luyện chủ yếu trên dữ liệu tiếng Anh hoặc dữ liệu đa ngôn ngữ nhưng có tỷ trọng tiếng Anh chiếm ưu thế. Điều này dẫn đến một số hệ quả khi áp dụng cho tiếng Việt trong miền tâm lý:

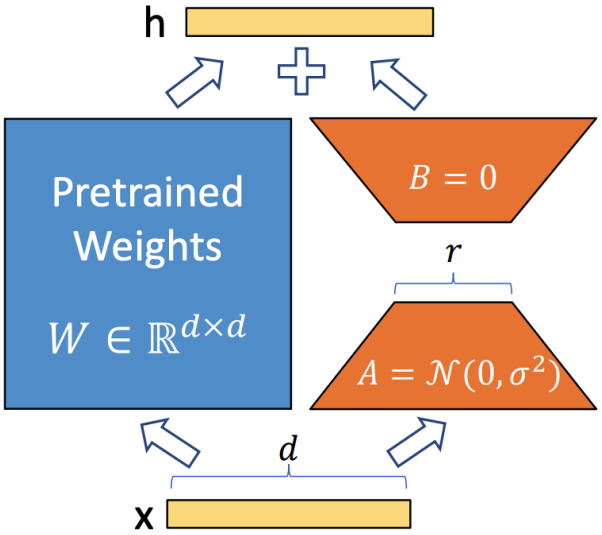
- Hạn chế trong việc hiểu các biểu đạt cảm xúc mang tính ẩn dụ, nói giảm nói tránh hoặc từ lóng đặc trưng văn hóa Việt Nam.
- Khả năng nhận diện sắc thái cảm xúc tinh vi (ví dụ: chán nản kéo dài, mệt mỏi tồn tại, cảm giác vô nghĩa) chưa đủ chính xác nếu không có dữ liệu tinh chỉnh chuyên biệt.
- Thiếu kiến thức hệ thống hóa về các khái niệm tâm lý học được diễn đạt bằng thuật ngữ tiếng Việt chuẩn hóa.

Do đó, trong hệ thống ViMind, quá trình fine-tuning không chỉ là một bước cải tiến tùy chọn mà mang tính bắt buộc nhằm căn chỉnh hành vi của mô hình cơ sở theo ba hướng chính:

- **Căn chỉnh theo miền tri thức :**
  Mô hình được huấn luyện bổ sung trên bộ dữ liệu tâm lý tiếng Việt đã được chọn lọc và chuẩn hóa, bao gồm tài liệu học thuật, kỹ thuật can thiệp (CBT, DBT), tình huống mô phỏng và kịch bản tư vấn. Điều này giúp mô hình hiểu đúng thuật ngữ chuyên ngành và mối quan hệ khái niệm trong tâm lý học.
- **Căn chỉnh theo phong cách và đạo đức phản hồi:**
  Fine-tuning giúp mô hình hình thành phong cách phản hồi phù hợp với nguyên tắc tham vấn tâm lý: thấu cảm, trung lập, không phán xét, tránh khẳng định tuyệt đối và hạn chế đưa ra lời khuyên mang tính điều trị chuyên sâu.
- **Căn chỉnh theo đặc thù ngôn ngữ và văn hóa Việt Nam:**
  Việc huấn luyện bổ sung trên dữ liệu bản địa giúp mô hình thích nghi tốt hơn với cấu trúc câu linh hoạt của tiếng Việt, các tầng nghĩa ngầm và cách biểu đạt cảm xúc gián tiếp thường gặp trong giao tiếp đời sống.

Nhờ quá trình fine-tuning chuyên biệt này, mô hình trong ViMind không còn chỉ là một LLM tổng quát, mà trở thành một mô hình ngôn ngữ được chuyên môn hóa cho lĩnh vực tâm lý tiếng Việt. Điều này giúp nâng cao độ chính xác, tính an toàn và mức độ thấu cảm của phản hồi, đồng thời giảm thiểu rủi ro phát sinh từ việc sử dụng trực tiếp các Base LLMs được huấn luyện chủ yếu trên dữ liệu tiếng Anh.

2.3.2. Phương pháp Low-Rank Adaptation (LoRA)



Hình 3. Phương pháp LoRA

Thay vì tinh chỉnh toàn bộ tham số của mô hình (Full Fine-tuning) – một quá trình đòi hỏi tài nguyên tính toán và bộ nhớ phần cứng rất lớn – đề tài lựa chọn phương pháp Parameter-Efficient Fine-Tuning (PEFT), cụ thể là kỹ thuật LoRA (Low-Rank Adaptation) do Hu và các cộng sự (2021) [4] đề xuất.

LoRA được xây dựng trên giả thuyết rằng những thay đổi cần thiết trong quá trình tinh chỉnh mô hình thực chất có “hạng thấp nội tại” (low intrinsic rank). Nói cách khác, để thích nghi với một miền dữ liệu chuyên biệt, mô hình không cần cập nhật toàn bộ không gian tham số khổng lồ, mà chỉ cần điều chỉnh một tập con cấu trúc có tính nén cao.

Thay vì cập nhật trực tiếp toàn bộ trọng số đã được huấn luyện trước, LoRA giữ nguyên các tham số gốc của LLM và bổ sung các thành phần thích nghi có kích thước nhỏ vào một số lớp quan trọng trong kiến trúc Transformer, đặc biệt là các lớp Attention (như Query và Value). Các thành phần bổ sung này được biểu diễn dưới dạng các ma trận phân rã hạng thấp có thể huấn luyện. Trong quá trình huấn luyện, chỉ các ma trận này được cập nhật, trong khi phần lõi của mô hình vẫn được giữ nguyên.

Về bản chất, sự thay đổi trọng số của mô hình được biểu diễn thông qua một cấu trúc phân rã gọn nhẹ, giúp giảm đáng kể số lượng tham số cần tối ưu. Thay vì phải điều chỉnh hàng chục tỷ tham số như trong Full Fine-tuning, phương pháp LoRA chỉ yêu cầu cập nhật một lượng tham số nhỏ hơn rất

nhiều, thường chỉ ở mức vài triệu tham số. Điều này mang lại ba lợi ích quan trọng:

- Giảm đáng kể yêu cầu tài nguyên phần cứng, đặc biệt là dung lượng VRAM trong quá trình huấn luyện.
- Rút ngắn thời gian huấn luyện, giúp quá trình thử nghiệm và tinh chỉnh trở nên khả thi hơn.
- Tăng tính linh hoạt khi triển khai, do các trọng số LoRA (LoRA adapters) có thể được lưu trữ và hoán đổi độc lập mà không cần tải lại toàn bộ mô hình nền.

Trong hệ thống ViMind, mô hình nền có thể được lựa chọn linh hoạt tùy theo tài nguyên phần cứng sẵn có. Trong trường hợp có hạ tầng tính toán đủ mạnh, hệ thống có thể sử dụng mô hình quy mô lớn như GPT-OSS 20B nhằm tận dụng năng lực biểu diễn và suy luận cao hơn. Ngược lại, khi triển khai trên hạ tầng hạn chế tài nguyên, các mô hình nhẹ hơn như Qwen3-8B-Base hoặc Qwen3-4B-Base có thể được sử dụng làm mô hình nền để đảm bảo tính khả thi và hiệu quả chi phí.

Nhờ kết hợp LoRA với chiến lược lựa chọn mô hình nền phù hợp, hệ thống có thể đạt được sự cân bằng giữa hiệu năng, chi phí tài nguyên và khả năng mở rộng. Đồng thời, mỗi Agent trong kiến trúc Multi-Agent có thể được tinh chỉnh bằng adapter LoRA chuyên biệt, tạo nên sự chuyên môn hóa theo nhiệm vụ mà vẫn duy trì một lõi mô hình thống nhất.

3. Tech Stack

Để xây dựng một hệ thống phần mềm hoàn chỉnh, có khả năng mở rộng và đáp ứng được các yêu cầu khắt khe về xử lý ngôn ngữ tự nhiên như ViMind, kiến trúc tổng thể được phân rã thành 4 thành phần chính: Giao diện người dùng (Frontend), Máy chủ xử lý (Backend), Cơ sở dữ liệu (Database) và AI. Việc phân tách này giúp tối ưu hóa hiệu năng cho từng cụm tác vụ và tạo điều kiện thuận lợi cho quá trình bảo trì, nâng cấp hệ thống.

3.1. Giao diện người dùng

Phân hệ Frontend đóng vai trò là lớp tương tác trực tiếp giữa hệ thống và người dùng cuối, chịu trách nhiệm tiếp nhận truy vấn, hiển thị phản hồi từ AI và duy trì trải nghiệm hội thoại liền mạch. Trong một hệ thống tư vấn tâm lý như ViMind, giao diện không chỉ là lớp trình bày thông tin mà còn phải xử lý các yêu cầu về đồng bộ trạng thái phiên làm việc, cập nhật nội dung theo thời gian thực và đảm bảo tính ổn định khi số lượt trao đổi tăng lên.

Về nền tảng công nghệ, hệ thống sử dụng HTML để tổ chức cấu trúc tài liệu, CSS để kiểm soát bố cục và phong cách hiển thị, cùng JavaScript để xử lý logic phía máy khách. Tuy nhiên, để đáp ứng các yêu cầu về hiệu năng, khả năng mở rộng và quản lý trạng thái phức tạp, hệ thống lựa chọn Next.js[5] làm framework chủ đạo, được xây dựng trên thư viện React.

Để tối ưu hóa quá trình phát triển và tăng cường hiệu năng hoạt động, hệ thống sử dụng Next.js làm framework chủ đạo. Next.js là một framework phát triển dựa trên thư viện React, cung cấp các cơ chế kết xuất dữ liệu linh hoạt như Server-Side Rendering (SSR) và Static Site Generation (SSG). Việc áp dụng Next.js giúp ViMind giảm thiểu thời gian tải trang ban đầu (initial load time), tối ưu hóa luồng tương tác với người dùng trong giao diện không gian tư vấn tâm lý (chat interface), và quản lý trạng thái (state management) một cách nhất quán khi phải liên tục cập nhật các đoạn hội thoại streaming từ hệ thống AI.

3.2. Máy chủ xử lý

Phân hệ Backend của ViMind được thiết kế theo hướng kiến trúc vi hybrid microservices, trong đó các thành phần được tách biệt theo chức năng chuyên môn nhằm tối ưu hóa hiệu năng, khả năng mở rộng và độ linh hoạt khi triển khai thực tế. Thay vì tập trung toàn bộ logic vào một dịch vụ đơn khối, hệ thống chia tách các nhóm nhiệm vụ có tính chất khác biệt — quản lý nghiệp vụ nền tảng và xử lý AI hiệu năng cao — thành các dịch vụ riêng biệt nhưng phối hợp chặt chẽ thông qua giao tiếp API.

Toàn bộ Backend được xây dựng bằng ngôn ngữ Python nhằm tận dụng hệ sinh thái mạnh về xử lý dữ liệu, học máy và tích hợp mô hình ngôn ngữ lớn. Hai framework chính được sử dụng là Django và FastAPI, mỗi framework đảm nhiệm một vai trò rõ ràng trong kiến trúc tổng thể.

Django – Lớp điều phối nghiệp vụ và quản lý dữ liệu

Django được sử dụng làm framework cốt lõi để xây dựng nền tảng quản trị và điều phối nghiệp vụ của hệ thống [6]. Django hoạt động như “xương sống” của Backend, chịu trách nhiệm xử lý các logic nghiệp vụ có tính ổn định cao và yêu cầu tính toàn vẹn dữ liệu.

Cụ thể, Django đảm nhiệm:

- **Hệ thống xác thực và định danh người dùng:**
  Bao gồm đăng ký, đăng nhập, mã hóa mật khẩu, xác thực phiên và quản lý token.

- **Quản lý phiên làm việc:**
  Lưu trữ và truy vết trạng thái hội thoại, lịch sử trao đổi và các metadata liên quan.
- **Chuẩn hóa và lưu trữ dữ liệu:**
  Django ORM giúp ánh xạ mô hình dữ liệu Python sang hệ quản trị cơ sở dữ liệu quan hệ, đảm bảo tính toàn vẹn tham chiếu (referential integrity) và hỗ trợ migration linh hoạt khi thay đổi cấu trúc bảng.

Nhờ kiến trúc MVT (Model–View–Template), Django cho phép tổ chức mã nguồn theo cấu trúc rõ ràng, giảm sự phụ thuộc chéo giữa các lớp logic và hỗ trợ bảo trì lâu dài.

FastAPI – Cổng API hiệu năng cao cho AI Core

Song song với Django, hệ thống triển khai FastAPI như một dịch vụ chuyên biệt để xử lý các yêu cầu đòi hỏi tốc độ cao và khả năng bất đồng bộ [7]. Nếu Django đóng vai trò điều phối nghiệp vụ nền, thì FastAPI hoạt động như một API Gateway hướng AI.

Trong kiến trúc của ViMind, FastAPI đảm nhiệm:

- Chuyển tiếp yêu cầu đến hệ thống Multi-Agent RAG để thực hiện truy xuất và suy luận.
- Giao tiếp trực tiếp với máy chủ vLLM đang host các mô hình ngôn ngữ đã được finetune, ví dụ như GPT-OSS 20B hoặc các biến thể Qwen3.

Khả năng xử lý bất đồng bộ của FastAPI cho phép hệ thống duy trì nhiều kết nối đồng thời mà không chặn luồng xử lý chính. Điều này đặc biệt quan trọng trong bối cảnh hội thoại AI cần trả về phản hồi dạng streaming token-by-token. Nhờ đó, hệ thống có thể đảm bảo trải nghiệm đàm thoại mượt mà mà không gây tắc nghẽn cho Django — vốn tập trung vào quản lý dữ liệu và nghiệp vụ nền.

3.3. Cơ sở dữ liệu (Database)

3.3.1. Cơ sở dữ liệu quan hệ cho các tác vụ nền

Đối với các nghiệp vụ quản trị nền tảng như lưu trữ thông tin định danh, PostgreSQL được triển khai làm hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) cốt lõi [8].

Việc sử dụng PostgreSQL đảm bảo tính toàn vẹn của dữ liệu thông qua cơ chế tuân thủ nghiêm ngặt các tiêu chuẩn ACID (Atomicity, Consistency, Isolation, Durability).

3.3.2. Cơ sở dữ liệu Vector cho tác vụ AI và RAG

Khác với dữ liệu quan hệ, phân hệ AI Multi-Agent RAG yêu cầu một giải pháp chuyên biệt để lưu trữ và truy vấn các biểu diễn vector nhiều chiều được trích xuất từ kho tài liệu và cẩm nang tâm lý học tiếng Việt. Hệ thống sử dụng MongoDB để đảm nhiệm vai trò Vector Database [9].

Thông qua tính năng Vector Search, MongoDB cho phép lưu trữ linh hoạt các đoạn văn bản dưới dạng tài liệu JSON, đồng thời tích hợp trực tiếp các mảng vector nhúng tương ứng. Kiến trúc NoSQL này tối ưu hóa việc thực thi các thuật toán tìm kiếm lân cận gần nhất (như Approximate Nearest Neighbor - ANN) dựa trên các phép đo độ tương đồng ngữ nghĩa. Khả năng truy xuất tốc độ cao của MongoDB đóng vai trò quyết định giúp các Retrieval Agent trích xuất chính xác ngữ cảnh theo thời gian thực, cung cấp dữ kiện đầu vào chất lượng cho các mô hình ngôn ngữ lớn đưa ra lập luận tư vấn tâm lý.

3.4. AI Core

Phân hệ Trí tuệ nhân tạo là trung tâm xử lý nhận thức của hệ thống ViMind, đảm nhiệm việc truy xuất tri thức chuyên môn, phân tích tâm lý và sinh ngôn ngữ tự nhiên. Để tối ưu hóa hiệu năng phát triển và triển khai kiến trúc học sâu, hệ thống tích hợp các công nghệ và bộ khung (framework) chuyên biệt sau:

- **Khung điều phối hệ thống Đa tác tử (LangChain và LangGraph):** Hệ thống sử dụng LangChain làm bộ khung cơ sở để quản lý kết nối giữa các mô hình ngôn ngữ, mô hình nhúng (embedding) và cơ sở dữ liệu vector. Để giải quyết bài toán điều phối luồng giao tiếp phức tạp giữa nhiều Agent, LangGraph được áp dụng nhằm mô hình hóa hệ thống Multi-Agent dưới dạng đồ thị có trạng thái (stateful graph) [10]. Trong cấu trúc này, mỗi tác tử chuyên biệt đóng vai trò là một nút (node) xử lý. Quá trình kiểm tra chéo và luân chuyển thông tin được quản lý chặt chẽ thông qua các cạnh (edges) có điều kiện. Cơ chế này cho phép thực thi các chu trình lặp , giúp hệ thống tự động đánh giá và hiệu chỉnh câu trả lời trước khi đưa ra phản hồi cuối cùng.
- **Khung tối ưu hóa tinh chỉnh mô hình (Unsloth):** Đối với nhiệm vụ huấn luyện tinh chỉnh (finetuning) các mô hình ngôn ngữ nền tảng gốc bằng phương pháp LoRA, hệ thống dự kiến sử dụng nền tảng Unsloth

[11]. Nền tảng này ứng dụng các thuật toán tối ưu hóa toán học ở mức vi kiến trúc, cho phép tăng tốc độ quá trình huấn luyện lên nhiều lần và giảm thiểu đáng kể dung lượng bộ nhớ đồ họa (VRAM) yêu cầu. Khả năng tương thích tốt của Unsloth giúp dự án dễ dàng xuất các bộ trọng số (adapters) chuẩn hóa để tích hợp thẳng vào các engine phục vụ suy luận.

- **Đánh giá và đo lường hệ thống (RAGAS):** Nhằm định lượng hóa chất lượng của hệ thống RAG thay vì phụ thuộc hoàn toàn vào đánh giá thủ công, nền tảng RAGAS (Retrieval Augmented Generation Assessment) được sử dụng làm khung đo lường tự động [12]. RAGAS cung cấp các độ đo kỹ thuật chuyên sâu để đánh giá ba trụ cột cốt lõi của RAG: độ trung thực của câu trả lời so với tài liệu trích xuất (Faithfulness), mức độ giải quyết đúng trọng tâm truy vấn của người dùng (Answer Relevance), và độ chính xác của ngữ cảnh được truy xuất từ Vector Database (Context Precision/Recall).
- **Thu thập ngữ liệu:** Cơ sở tri thức tâm lý học Tiếng Việt được tổng hợp từ nhiều nguồn y tế, diễn đàn tư vấn và tài liệu học thuật. Quá trình thu thập tự động được triển khai bằng framework Scrapy[13] để tối ưu tốc độ cào dữ liệu tĩnh ở quy mô lớn, kết hợp cùng Selenium[14] để giả lập hành vi người dùng nhằm trích xuất các nội dung web động phức tạp.

4. Phân tích yêu cầu nghiệp vụ

Hệ thống cần đáp ứng các yêu cầu nghiệp vụ sau:

- Hệ thống cần quản lý thông tin tài khoản, lưu trữ lịch sử phiên làm việc và đảm bảo bảo mật tuyệt đối cho các dữ liệu cá nhân nhạy cảm của người dùng.
- Cung cấp không gian giao tiếp mượt mà, ổn định theo thời gian thực, cho phép người dùng trao đổi tự nhiên với hệ thống AI mà không bị gián đoạn.
- Xây dựng, phân loại và cập nhật liên tục hệ thống dữ liệu chuyên môn về tâm lý học từ các nguồn tài liệu uy tín để làm nền tảng tham chiếu cho AI.
- Hệ thống AI cần có khả năng tiếp nhận vấn đề, phân tích ngữ cảnh, tra cứu thông tin và tự động kiểm tra chéo kết quả để đưa ra lời khuyên tâm lý chính xác, có cơ sở.
- Cần có cơ chế theo dõi và đo lường tự động mức độ tin cậy, độ chuẩn xác của các câu trả lời do AI tạo ra để ngăn ngừa rủi ro cung cấp thông tin sai lệch.

5. Kế hoạch xây dựng và triển khai hệ thống

Công việc thực hiện	Quãng thời gian
Xử lý dữ liệu: Thu thập, làm sạch và chuẩn hóa bộ dữ liệu tâm lý học Tiếng Việt từ các nguồn.	1 tuần
Phân tích và thiết kế: Phân tích yêu cầu nghiệp vụ, thiết kế kiến trúc hệ thống vi dịch vụ, và thiết kế cơ sở dữ liệu.	1 tuần
Phát triển Backend: Xây dựng hệ thống quản trị nền tảng và thiết lập cổng giao tiếp API bất đồng bộ.	1 tuần
Phát triển lõi AI (RAG): Cấu hình Vector Database, thiết kế luồng và xây dựng các Agent bằng LangChain/LangGraph.	2 tuần
Tinh chỉnh mô hình (Finetuning): Tinh chỉnh LLM bằng kỹ thuật LoRA, đánh giá ban đầu và tích hợp mô hình vào engine suy luận.	2 tuần
Phát triển Frontend: Xây dựng giao diện người dùng (Next.js), xử lý luồng streaming hội thoại và ghép nối toàn diện với các API Backend.	1 tuần

Kiểm thử và triển khai: Đánh giá chất lượng RAG bằng RAGAS, kiểm thử tải hệ thống, sửa lỗi và triển khai bản hoàn thiện.	1 tuần

6. Các kỹ thuật phát triển bổ sung

Trong trường hợp khả thi về mặt thời gian và tài nguyên, dự án sẽ được mở rộng nghiên cứu và triển khai thêm hai kỹ thuật sau nhằm tối ưu hóa hiệu năng và trải nghiệm người dùng:

- **Nghiên cứu và tích hợp GraphRAG:** Hệ thống dự kiến mở rộng kết hợp cơ sở dữ liệu đồ thị tri thức bên cạnh Vector Database hiện tại. Kỹ thuật này giúp lập bản đồ các mối quan hệ phức tạp giữa các khái niệm tâm lý học, nhóm triệu chứng và phương pháp can thiệp. Điều này giúp hệ thống khắc phục các điểm mù truy xuất và nâng cao độ chính xác khi phải suy luận qua nhiều bước.
- **Triển khai mô hình nhân vật ảo:** Nhằm tăng cường tính thấu cảm trong quá trình tham vấn, hệ thống dự kiến tích hợp giao diện nhân vật tư vấn ảo. Nhân vật này sẽ được đồng bộ hóa trực tiếp với luồng xử lý ngôn ngữ tự nhiên của AI, cho phép tạo ra các biểu cảm và cử chỉ giao tiếp phù hợp với ngữ cảnh cảm xúc của cuộc hội thoại, mang lại trải nghiệm sinh động và thân thiện hơn cho người dùng.

References

[1] P. Lewis *et al*., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[2] L. Wang *et al*., “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, Art. no. 186302, 2024. doi: 10.1007/s11704-024-40231-1.

[3] Q. Wu *et al*., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” arXiv:2308.08155, Aug. 2023.

[4] E. J. Hu *et al*., “LoRA: Low-Rank Adaptation of Large Language Models,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2022.

[5] Vercel, “Next.js: The React Framework for the Web,” 2023. [Online]. Available: https://nextjs.org/. [Accessed: Feb. 26, 2026].

[6] Django Software Foundation, “Django documentation, release 6.0,” 2024. [Online]. Available: https://docs.djangoproject.com/en/6.0/. [Accessed: Feb. 26, 2026].

[7] S. Ramírez, “FastAPI: High performance, easy to learn, fast to code, ready for production,” 2018. [Online]. Available: https://fastapi.tiangolo.com/. [Accessed: Feb. 26, 2026].

[8] PostgreSQL Global Development Group, “PostgreSQL: The World's Most Advanced Open Source Relational Database,” 2023. [Online]. Available: https://www.postgresql.org/. [Accessed: Feb. 26, 2026].

[9] MongoDB Inc., “MongoDB: The Developer Data Platform,” 2023. [Online]. Available: https://www.mongodb.com/. [Accessed: Feb. 26, 2026].

[10] LangChain, “LangGraph: Multi-Actor, Stateful LLM Applications,” 2024. [Online]. Available: https://langchain-ai.github.io/langgraph/. [Accessed: Feb. 26, 2026].

[11] Unsloth AI, “Unsloth: Finetune Llama, Mistral, Gemma 2x–5x faster with 70% less memory,” 2024. [Online]. Available: https://unsloth.ai/. [Accessed: Feb. 26, 2026].

[12] S. Es *et al.*, “RAGAS: Automated Evaluation of Retrieval Augmented Generation,” arXiv:2309.15217, Sep. 2023.

[13] Scrapy Project, “Scrapy documentation (latest),” [Online]. Available: https://docs.scrapy.org/en/latest/. [Accessed: Feb. 26, 2026].

[14] SeleniumHQ, “Selenium documentation,” [Online]. Available: https://www.selenium.dev/documentation/. [Accessed: Feb. 26, 2026].